

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П.Королева»
(Самарский университет)

ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ № 7

«Работа с паттернами и рефлексией»

по курсу
Объектно-ориентированное программирование

Выполнила: Гонтарь Анастасия Вячеславовна,
студент группы 6203-010302D

Содержание

<u>Задание №1</u>	3
<u>Задание №2</u>	5
<u>Задание №3</u>	7

Задание №1

Чтобы объекты табулированной функции можно было использовать в for-each сначала я добавила наследование :

```
public interface TabulatedFunction extends Iterable<FunctionPoint>, Function, Cloneable { 28 usages 2 implementations
```

а следом реализовала итераторы для ArrayTabulatedFunction

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0; // текущий индекс для итерации 3 usages

        @Override
        public boolean hasNext() { // есть ли следующая точка
            return index < pointsCount;
        }

        @Override
        public FunctionPoint next() { // возвращает следующую точку
            if (!hasNext()) {
                throw new NoSuchElementException("следующей точки не существует");
            }
            FunctionPoint point = points[index];
            FunctionPoint copyPoint = new FunctionPoint(point.getX(), point.getY()); // создаем копию точки
            index++;
            return copyPoint;
        }

        @Override
        public void remove() { // удаление точки не предусмотрено
            throw new UnsupportedOperationException("удаление невозможно");
        }
    };
};
```

и LinkedListTabulatedFunction

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode curr = head.getNext(); 4 usages

        @Override
        public boolean hasNext() {
            return curr != head;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("следующего элемента не существует");
            }
            FunctionPoint point = new FunctionPoint(curr.getPoint());
            curr = curr.getNext();
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("удаление невозможно");
        }
    };
};
```

Оба итератора используют знания о внутренней структуре и возвращают копию точек для сохранения инкапсуляции.

Далее в Main я протестировала написанный код. Вывела при помощи цикла for-each array и linked list

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Задание 1 \n");  
        TabulatedFunction arrFunc = new ArrayTabulatedFunction( leftX: 0, rightX: 5, pointsCount: 6);  
        TabulatedFunction linkFunc = new LinkedListTabulatedFunction( leftX: 0, rightX: 5, pointsCount: 6);  
        System.out.println("проверка работы вывода массива \n");  
        for (FunctionPoint p : arrFunc) {  
            System.out.println(p);  
        }  
        System.out.println("\nпроверка работы вывода списка \n");  
        for (FunctionPoint p : linkFunc) {  
            System.out.println(p);  
        }  
    }  
}
```

Задание 1

проверка работы вывода массива

(0,0; 0,0)
(1,0; 0,0)
(2,0; 0,0)
(3,0; 0,0)
(4,0; 0,0)
(5,0; 0,0)

проверка работы вывода списка

(0,0; 0,0)
(1,0; 0,0)
(2,0; 0,0)
(3,0; 0,0)
(4,0; 0,0)
(5,0; 0,0)

Задание №2

Для выполнения этого задания нужно было воспользоваться паттерном “Фабричный метод”

Создала интерфейс:

```
1 package functions;
2
3 public interface TabulatedFunctionFactory { 4 usages 2 implementations
4     public TabulatedFunction createTabulatedFunction(FunctionPoint[] values); 1 usage 2 implementations
5     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount); 1 usage 2 implementations
6     public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values); 1 usage 2 implementations
7 }
8
```

В классах array и linked list я реализовала вложенные классы фабрик, реализующих интерфейс фабрики и порождающих объекты соответствующих классов табулированных функций:

```
// фабрика
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { 2 usages
    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

```
// фабрика
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory{ 1 usage

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override 1 usage
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

В классе TabulatedFunctions я объявила и описала все необходимые методы

```
private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); 4 usages

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] values) { 2 usages
    return factory.createTabulatedFunction(values);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { 1 usage
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usages
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory f) { 2 usages
    factory = f;
}
```

и в остальных методах класса, где требуется создание объектов табулированных функций, заменила явное создание объектов с помощью конструкторов на вызов соответствующего метода createTabulatedFunction().

На рисунке представлен пример замены в методе tabulate

```
public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) { 3 usages
    if (function == null)
        throw new IllegalArgumentException("функция null");
    if (pointsCount < 2) {
        throw new IllegalArgumentException("недостаточное количество точек");
    }
    if (leftX >= rightX) {
        throw new IllegalArgumentException("левая граница меньше правой");
    }
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("границы табулирования выходят за область определения");
    }

    TabulatedFunction tabulatedFunc = createTabulatedFunction(leftX, rightX, pointsCount);
    // записываем значения в массив
    for (int i = 0; i < pointsCount; i++){
        double x = tabulatedFunc.getPointX(i);
        double y = function.getFunctionValue(x);
        tabulatedFunc.setPointY(i, y);
    }

    return tabulatedFunc;
}
```

Вывод проверки работы фабрик, предложенным кодом

Задание 2

```
class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction
```

Задание №3

В этом задании я реализовала использование рефлексии, что позволит выбирать какой класс использовать для создания табулированной функции. Перегрузила методы, добавив проверки

```
// РЕФЛЕКСИЯ
public static TabulatedFunction createTabulatedFunction(Class<?> funcClass, double leftX, double rightX, int pointsCount) { 2 usages
    try {
        if (!TabulatedFunction.class.isAssignableFrom(funcClass)) { // проверяем, что класс реализует TabulatedFunction
            throw new IllegalArgumentException("класс не реализует интерфейс TabulatedFunction");
        }
        Constructor<?> constructor = funcClass.getConstructor(double.class, double.class, int.class); // ищем нужный конструктор

        return (TabulatedFunction) constructor.newInstance(leftX, rightX, pointsCount); // создаем объект через рефлексию
    } catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException | InstantiationException e) {
        throw new IllegalArgumentException("ошибка при создании объекта", e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<?> funcClass, double leftX, double rightX, double[] values) { 1 usage
    try {
        if (!TabulatedFunction.class.isAssignableFrom(funcClass)) { // проверяем, что класс реализует TabulatedFunction
            throw new IllegalArgumentException("класс не реализует интерфейс TabulatedFunction");
        }
        Constructor<?> constructor = funcClass.getConstructor(double.class, double.class, double[].class); // ищем нужный конструктор

        return (TabulatedFunction) constructor.newInstance(leftX, rightX, values); // создаем объект через рефлексию
    } catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException | InstantiationException e) {
        throw new IllegalArgumentException("ошибка при создании объекта", e);
    }
}
}
```

```
public static TabulatedFunction createTabulatedFunction(Class<?> funcClass, FunctionPoint[] points) { 3 usages
    try {
        if (!TabulatedFunction.class.isAssignableFrom(funcClass)) { // проверяем, что класс реализует TabulatedFunction
            throw new IllegalArgumentException("класс не реализует интерфейс TabulatedFunction");
        }
        Constructor<?> constructor = funcClass.getConstructor(FunctionPoint[].class); // ищем нужный конструктор

        return (TabulatedFunction) constructor.newInstance((Object) points); // создаем объект через рефлексию
    } catch (NoSuchMethodException | IllegalAccessException | InvocationTargetException | InstantiationException e) {
        throw new IllegalArgumentException("ошибка при создании объекта", e);
    }
}
}
```

А также перегрузила методы, создающие объекты табулированной функции, например, метод `tabulate`

```
public static TabulatedFunction tabulate(Class<?> funcClass, Function function, double leftX, double rightX, int pointsCount) { 1 usage
    if (function == null)
        throw new IllegalArgumentException("функция null");
    if (pointsCount < 2) {
        throw new IllegalArgumentException("недостаточное количество точек");
    }
    if (leftX >= rightX) {
        throw new IllegalArgumentException("левая граница меньше правой");
    }
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("границы табулирования выходят за область определения");
    }

    TabulatedFunction tabulatedFunc = createTabulatedFunction(funcClass, leftX, rightX, pointsCount);
    // записываем значения в массив
    for (int i = 0; i < pointsCount; i++){
        double x = tabulatedFunc.getPointX(i);
        double y = function.getFunctionValue(x);
        tabulatedFunc.setPointY(i, y);
    }

    return tabulatedFunc;
}
```

Проверила работу метода, предложенным кодом

Задание 3

```
class functions.ArrayTabulatedFunction
{(0,0; 0,0)(5,0; 0,0)(10,0; 0,0)}
class functions.ArrayTabulatedFunction
{(0,0; 0,0)(10,0; 10,0)}
class functions.LinkedListTabulatedFunction
{(0,0; 0,0)(10,0; 10,0)}
class functions.LinkedListTabulatedFunction
{(0,0; 0,0)(0,3; 0,3)(0,6; 0,6)(0,9; 0,8)(1,3; 1,0)(1,6; 1,0)(1,9; 1,0)(2,2; 0,8)(2,5; 0,6)(2,8; 0,3)(3,1; 0,0)}
```